

Hamiltonian Normalizing Flows as kinetic PDE solvers: application to the 1D Vlasov-Poisson Equations

Vincent Souveton

Sébastien Terrana

CEA, DAM, DIF, Arpajon, F-91297, France

VINCENT.SOUVETON@CEA.FR

SEBASTIEN.TERRANA@CEA.FR

Editors: Cecília Coelho, Bernd Zimmering, M. Fernanda P. Costa, Luís L. Ferrás, Oliver Niggemann

Abstract

Many conservative physical systems can be described using the Hamiltonian formalism. A notable example are the Vlasov-Poisson equations, a set of partial differential equations that govern the time evolution of a phase-space density function representing collisionless particles under a self-consistent potential. These equations play a central role in both plasma physics and cosmology. Due to the complexity of the potential involved, analytical solutions are rarely available, necessitating the use of numerical methods such as Particle-In-Cell. In this work, we introduce a novel approach based on Hamiltonian-informed Normalizing Flows, specifically a variant of Fixed-Kinetic Neural Hamiltonian Flows. Our method transforms an initial Gaussian distribution in phase space into the final distribution using a sequence of invertible, volume-preserving transformations derived from Hamiltonian dynamics. The model is trained on a dataset comprising initial and final states at a fixed time T , generated via numerical simulations. After training, the model enables fast sampling of the final distribution from any given initial state. Moreover, by automatically learning an interpretable physical potential, it can generalize to intermediate states not seen during training, offering insights into the system’s evolution across time.

Keywords: Machine Learning, Normalizing Flows, Physics-Informed Neural Networks, Neural Hamiltonian Flows, Kinetic Equations, Vlasov-Poisson Equations

1. Introduction

Complex particle systems arise across a wide range of physical domains, including plasma dynamics and cosmology. In the mean-field limit, where the number of interacting particles becomes large, these systems are often described by kinetic partial differential equations (PDEs). Such equations are typically derived as formal limits of the underlying microscopic dynamics and characterize the time evolution of the system’s phase-space distribution. In this work, we focus on systems governed by Hamiltonian dynamics (Goldstein et al., 2002; Arnold, 2013), which are characterized by symplectic transformations that conserve phase-space volume and total energy over time. A prominent example is the Vlasov-Poisson system, which governs the dynamics of a distribution function under self-consistent fields and plays a central role in modeling both collisionless plasmas and the formation of large-scale cosmic structures (Nicholson and Nicholson, 1983; Bernardeau et al., 2002).

Due to the high dimensionality and nonlinear nature of these equations, analytical solutions are often intractable, and numerical simulations become essential. However, direct N -body simulations, which compute all pairwise interactions explicitly, scale with a complexity of $\mathcal{O}(N^2)$, rendering them impractical for large-scale systems (Hockney and

Eastwood, 1988). Particle-In-Cell (PIC) methods offer an efficient alternative by interpolating particle properties onto a grid to solve the field equations, reducing the complexity to $\mathcal{O}(N + N_g \ln N_g)$, where N_g is the number of grid points (Birdsall and Langdon, 1991). Widely-used PIC-based simulation codes include WarpX (Vay et al., 2018), OSIRIS (Fonseca et al., 2002), and Smilei (Derouillat et al., 2018). Developments have shown that machine learning (ML) techniques can enhance PIC simulations by efficiently computing the electric field on discretized grids (Aguilar and Markidis, 2021), thus keeping the need for a binning strategy.

More generally, the use of machine learning to accelerate simulations has gained significant interest. A key strategy to improve physical consistency is to embed domain knowledge directly into the learning process. Physics-Informed Neural Networks (Raissi et al., 2019), which minimize PDE residuals as part of the loss, exemplify this approach and have been applied to Vlasov–Poisson systems (Zhang et al., 2023). Also, probabilistic generative models such as Normalizing Flows (Papamakarios et al., 2021), Generative Adversarial Networks (Goodfellow et al., 2014), and Diffusion Models (Sohl-Dickstein et al., 2015; Ho et al., 2020) serve as effective surrogates for simulating complex dynamical systems and have been used in cosmic web emulation (Rodriguez et al., 2018), high-energy physics (Paganini et al., 2018), and plasma dynamics (Grathwohl et al., 2019; Del-Castillo-Negrete et al., 2023) for instance. For Hamiltonian systems, Hamiltonian Neural Networks (Greydanus et al., 2019) and Hamiltonian Generative Networks (Toth et al., 2020) yield physically plausible trajectories by learning the underlying Hamiltonian, though they are not directly suited for modeling full probability densities. Neural Hamiltonian Flows (NHF) (Toth et al., 2020) combine generative modeling with symplectic structure, enabling interpretable high-dimensional sampling (Souveton et al., 2024). Extensions incorporating symmetry-preserving biases directly into the loss have been proposed in Rezende et al. (2019).

In this paper, we introduce a novel Hamiltonian-based Normalizing Flows framework for simulating kinetic equations, termed **PDE-NHF** (*kinetic PDEs with NHF*). Our contributions are threefold. First, we propose a robust, physics-informed model trained with a probabilistic loss that tracks the evolution of the phase-space distribution, rather than individual particle trajectories. The method leverages Hamiltonian priors without requiring explicit governing equations. Second, we design a self-consistent, interpretable potential energy function that enforces physical symmetries through invariance under transformation groups without modifying the loss function. Third, through a one-dimensional experiment, we show that PDE-NHF, trained only on initial and final states, accurately recovers the full Vlasov–Poisson dynamics without intermediate supervision. The paper is structured as follows. Section 2 reviews the foundations of Hamiltonian-based Normalizing Flows. Section 3 introduces the Vlasov–Poisson system and the PDE-NHF model. Section 4 evaluates PDE-NHF on a one-dimensional benchmark. Section 5 concludes with a discussion of results and future directions.

2. Hamiltonian-based Normalizing Flows

2.1. Hamiltonian mechanics

Hamiltonian mechanics is a reformulation of classical mechanics that describes the evolution of a system in terms of its generalized coordinates: positions $q \in \mathbb{R}^d$ and momenta $p \in \mathbb{R}^d$.

The state of the system $(q(t), p(t))$ evolves according to a set of first-order differential equations, known as Hamilton’s equations, which are applied componentwise:

$$\frac{dq}{dt} = \frac{\partial H}{\partial p}, \quad \frac{dp}{dt} = -\frac{\partial H}{\partial q}.$$

Here, $H = H(q, p)$ is the Hamiltonian function, typically representing the total energy of the system. In many cases, it can be decomposed into the sum of a potential energy term $V(q)$, depending only on the positions and a kinetic energy term $K(p)$, depending only on the momenta:

$$H(q, p) = V(q) + K(p).$$

Starting from an initial condition (q_0, p_0) in phase space, Hamilton’s equations define a unique flow $\mathcal{T}^t$ that evolves the state to $(q(t), p(t))$ at any future time $t > 0$. This flow possesses several fundamental properties: it is **time reversible** as running the system backward in time exactly retraces its trajectory; it **conserves the energy** as the Hamiltonian remains constant along trajectories if it does not explicitly depend on time; finally, the flow is **symplectic**, meaning that it preserves the form $\omega = dq \wedge dp$. This implies that the volume in phase space is conserved, a property formalized by Liouville’s theorem (Landau and Lifshitz, 1976) $\frac{d}{dt} \det \left(\frac{\partial(q(t), p(t))}{\partial(q_0, p_0)} \right) = 0$. Therefore, the Jacobian determinant of the transformation is exactly equal to one.

2.2. Normalizing Flows

Normalizing Flows (Tabak and Vanden-Eijnden, 2010; Dinh et al., 2015; Rezende and Mohamed, 2015; Papamakarios et al., 2021) are generative models that transform a base distribution $\pi_0 \in \mathcal{C}^\infty(\mathbb{R}^d)$ into the target $\pi \in \mathcal{C}^\infty(\mathbb{R}^d)$ through a finite sequence of smooth, invertible transformations $\mathcal{T}_\theta = \mathcal{T}_\theta^L \circ \dots \circ \mathcal{T}_\theta^1$. Each $\mathcal{T}_\theta^\ell$, for $1 \leq \ell \leq L$, is a learned bijective map, and θ denotes the model parameters. Given a latent variable $Z \sim \pi_0$, the output variable is obtained as $Q = \mathcal{T}_\theta(Z)$. Using the change of variables, the model density reads $\pi_\theta(q) = \pi_0(\mathcal{T}_\theta^{-1}(q)) \times \prod_{\ell=1}^L \left| \det \text{Jac}_{(\mathcal{T}_\theta^\ell)^{-1}}(q) \right|$, where $\text{Jac}_{(\mathcal{T}_\theta^\ell)^{-1}}$ is the Jacobian matrix of the inverse transformation at step ℓ . The model is trained by minimizing the forward Kullback–Leibler divergence between π and π_θ :

$$\mathcal{L}(\theta) = \mathbb{E}_{q \sim \pi} [\ln \pi(q) - \ln \pi_\theta(q)] = -\mathbb{E}_{q \sim \pi} \left[\ln \pi_0(\mathcal{T}_\theta^{-1}(q)) + \sum_{\ell=1}^L \ln \left| \det \text{Jac}_{(\mathcal{T}_\theta^\ell)^{-1}}(q) \right| \right] + C,$$

where C is a constant. The expectation is approximated via Monte Carlo sampling. Designing flow-based models requires choosing smooth, invertible transformations whose Jacobian determinant is easy to compute. The resulting chain should be expressive and interpretable.

2.3. Neural Hamiltonian Flows

Neural Hamiltonian Flows (NHF) (Toth et al., 2020) are Physics-informed models that treat volume and energy-preserving, invertible, smooth and interpretable Hamiltonian transformations as normalizing flows. In training mode, a point q is sampled from the training

dataset. It is seen as a position vector associated to a single particle in dimension d . To simulate Hamiltonian dynamics, one needs to add artificial momenta p . This is achieved via two neural networks μ_θ and σ_θ that are used to sample $p \sim p_\theta(q) = \mathcal{N}(\mu_\theta(q), \sigma_\theta(q)^2)$. Then, one defines a kinetic energy $K_\theta(p)$ as well as a potential energy $V_\theta(q)$. In [Souveton et al. \(2024\)](#), the authors propose to fix the kinetic term to a quadratic form as in classical mechanics in order to improve interpretability and reduce the model complexity. In the resulting Fixed-Kinetic Neural Hamiltonian Flows (FKNHF) model, one sets $K_\theta(p) = \frac{1}{2}p\mathcal{M}_\theta^{-1}p^T$ with $\mathcal{M}_\theta^{-1}$ a learned positive-definite mass matrix. As for the potential energy V_θ , it is modeled as a neural network. System evolves in phase space following Hamilton’s equations approximated by an exactly volume-preserving Leapfrog integrator for L steps with timestep δt and initial conditions $(q(0), p(0))$. One step of the Leapfrog scheme reads:

$$\begin{cases} p\left(t + \frac{\delta t}{2}\right) = p(t) - \frac{\delta t}{2}\nabla_q V_\theta(q(t)), \\ q(t + \delta t) = q(t) + \delta t p\left(t + \frac{\delta t}{2}\right)\mathcal{M}_\theta^{-1} \\ p(t + \delta t) = p\left(t + \frac{\delta t}{2}\right) - \frac{\delta t}{2}\nabla_q V_\theta(q(t + \delta t)). \end{cases}$$

Overall, the model parametrizes a transformation $\mathcal{T}_\theta : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}^d \times \mathbb{R}^d$. Training involves maximizing an evidence lower bound ([Toth et al., 2020](#)) – see Appendix A:

$$\mathcal{L}_{\text{NHF}}(q; \theta) = \mathbb{E}_{p \sim p_\theta(q)} [\ln \Pi_0(\mathcal{T}_\theta^{-1}(q, p)) - \ln p_\theta(p|q)].$$

After training, the generative process consists in drawing $(q(0), p(0)) \sim \Pi_0$, applying L Leapfrog step with timestep $-\delta t$ and marginalizing over the momenta in the final state.

3. Solving Hamiltonian kinetic equations with PDE-NHF

3.1. Vlasov-Poisson system

Hamiltonian kinetic partial differential equations describe the temporal evolution of a probability density function $\Pi(q, p, t)$ in phase space, under the action of a Hamiltonian flow. One of the most fundamental examples is the **Vlasov equation**, which, in one dimension, reads:

$$\frac{\partial \Pi}{\partial t} + p \frac{\partial \Pi}{\partial q} + F(q, t) \frac{\partial \Pi}{\partial p} = 0.$$

This equation governs the evolution of a collisionless many-particle system subjected to a force field $F(q, t)$. The structure follows directly from Liouville’s theorem, which asserts that Π is conserved along the phase-space trajectories defined by Hamilton’s equations. In many applications, the force F depends on the evolving distribution of particles, leading to a self-consistent coupling between the Vlasov equation and a field equation. Typically, the force derives from a potential as $F(q, t) = -\frac{\partial V(q, t)}{\partial q}$ and the potential V itself satisfies a second-order **Poisson equation**. For instance, in the case of electrostatic systems:

$$\frac{\partial^2 V}{\partial q^2} = -\frac{\rho(q, t)}{\varepsilon_0},$$

where ε_0 is the permittivity of the medium, and $\rho(q, t) = \int \Pi(q, p, t) dp$ is the charge density associated with the distribution of charged particles. Thus, the Vlasov–Poisson system models the self-consistent evolution of particle ensembles under their collective interactions, while preserving the symplectic structure of Hamiltonian dynamics.

3.2. Introducing PDE-NHF

Context. Since the goal is to learn a Hamiltonian mapping between two distributions in phase space, one can reinterpret Hamiltonian-based Normalizing Flows as solvers for Hamiltonian kinetic PDEs. In the following, $\mathbf{q} := (q_1, \dots, q_N) \in \mathbb{R}^{N \times d}$ and $\mathbf{p} := (p_1, \dots, p_N) \in \mathbb{R}^{N \times d}$ denote the generalized positions and momenta, respectively, of a N -particle system living in a d -dimensional space. We are given a collection of initial Gaussian distributions $\Pi_0^\mathbf{x}$ in phase space, each parametrized by a mean and covariance matrix encoded through a vector $\mathbf{x} \in \mathcal{D}$. For each initial distribution, we are provided with a final set of N particle positions $\mathbf{q}_T$ and momenta $\mathbf{p}_T$ representing the distribution at a fixed time $T > 0$, obtained by evolving the initial distribution according to some Hamiltonian dynamics. We aim to train a model that maps any initial Gaussian distribution $\Pi_0^\mathbf{x}$, $\mathbf{x} \in \mathcal{D}$, to the corresponding final distribution $\Pi_T^\mathbf{x}$. Moreover, the model should be capable of reconstructing physically consistent intermediate distributions $\Pi_t^\mathbf{x}$ for all $0 < t < T$.

Solution. We propose a new generative model named **PDE-NHF** whose illustration can be found in Figure 1, derived from the FKNHF framework. A public version of the implementation is available at <https://github.com/vincentsovveton/PDE-NHF>.

Architecture. The total energy of the N -particle system used to approximate the phase-space distribution is the sum of a quadratic kinetic term $K_\theta(\mathbf{p})$ and a self-consistent potential energy term $V_\theta(\mathbf{q})$ designed to be invariant under permutations and translations of particles. We achieve this by defining V_θ as a Deep Set network (Zaheer et al., 2017). Specifically,

$$K_\theta(\mathbf{p}) = \frac{a^2}{2} \sum_{j=1}^N \|p_j\|^2, \quad V_\theta(\mathbf{q}) = \psi \left(\sum_{j=1}^N \phi \left(q_j - \frac{1}{N} \sum_{l=1}^N q_l \right) \right),$$

where a is a learned scalar parameter and ψ and ϕ are two neural networks. The system evolves using a Leapfrog scheme, thus ensuring that volume preservation is embedded into the architecture rather than enforced through the loss function. Since the Jacobian determinant of the induced transformation $\mathcal{T}_\theta^t : \mathbb{R}^{N \times d} \times \mathbb{R}^{N \times d} \rightarrow \mathbb{R}^{N \times d} \times \mathbb{R}^{N \times d}$ equals one, the model distribution is tractable at any time $0 \leq t \leq T$: $\Pi_{\theta,t}^\mathbf{x}(\mathbf{q}_t, \mathbf{p}_t) = \Pi_0^\mathbf{x}((\mathcal{T}_\theta^t)^{-1}(\mathbf{q}_t, \mathbf{p}_t))$.

Training. Contrary to FKNHF, as we are working in full phase space, there is no need to draw artificial momenta and the goal is to **minimize** the Kullback–Leibler divergence between the target and the model distributions, conditioned on the initial distribution parameters encoded in $\mathbf{x}$ which is uniformly drawn in $\mathcal{D}$. Given data $\{(\mathbf{q}_T^{(m)}, \mathbf{p}_T^{(m)}, \mathbf{x}^{(m)})\}_{m=1}^M$, the training procedure consists in mapping the final distribution onto the corresponding

initial Gaussian distribution. The loss function derived in Appendix A reads:

$$\begin{aligned}\mathcal{L}_{\text{PDE-NHF}}(\theta) &= -\mathbb{E}_{(\mathbf{q}_T, \mathbf{p}_T) \sim \Pi_T^*, \mathbf{x} \sim \mathcal{U}(\mathcal{D})} [\ln \Pi_0^* ((\mathcal{T}_\theta^T)^{-1}(\mathbf{q}_T, \mathbf{p}_T))] \\ &\approx -\frac{1}{M} \sum_{m=1}^M \ln \Pi_0^{*(m)} \left((\mathcal{T}_\theta^T)^{-1}(\mathbf{q}_T^{(m)}, \mathbf{p}_T^{(m)}) \right).\end{aligned}$$

Sampling. This is achieved by simulating the reversed learned Hamiltonian dynamics. It suffices to draw a Gaussian initial distribution of particles whose mean and covariance matrices are encoded through $\mathbf{x} \in \mathcal{D}$ and reverse the timestep sign in the Leapfrog scheme.

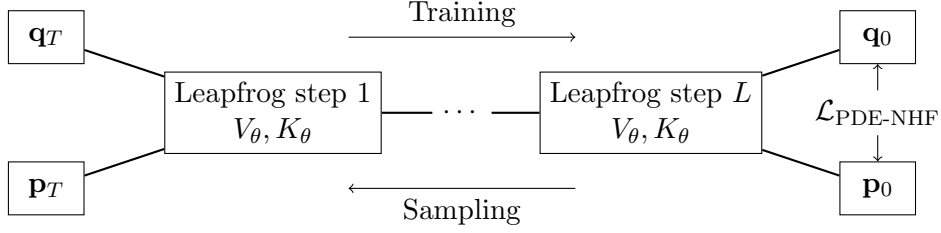


Figure 1: Schematic representation of PDE-NHF. Training consists in mapping a final distribution in phase space onto the correct initial distribution, without intermediate supervision. The learned reversed transformation allows one to sample from any initial Gaussian distribution and map it onto the corresponding final distribution while recovering intermediate timesteps.

4. Numerical application to the 1D Vlasov-Poisson equation

We build a dataset for learning a 1D Vlasov-Poisson dynamics made of 32768 examples. 20,000 examples are used for training, 6,384 for validation and 6,384 for testing. Each example consists in tracking the phase-space evolution of a 256-particle system in dimension $d = 1$ at times $t \in \{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ with a PIC method – see Appendix B. The performance of PDE-NHF are compared to that of a baseline MLP-based integrator – see Appendix C. As a metric, we use the one-dimensional Wasserstein distance W_1 . Given two samples $\mathbf{a} = (a_1, \dots, a_N)$ and $\mathbf{b} = (b_1, \dots, b_N)$, it is computed by first sorting the samples such that $a_{(1)} \leq \dots \leq a_{(N)}$ and $b_{(1)} \leq \dots \leq b_{(N)}$, and then evaluating $W_1(\mathbf{a}, \mathbf{b}) = \frac{1}{N} \sum_{j=1}^N |b_{(j)} - a_{(j)}|$. We summarize our findings and observations in the following conclusions.

1. First, **our model serves as an efficient integrator** by accurately sampling the correct final distribution in phase space. As shown in Table 1, PDE-NHF significantly outperforms the baseline integrator. The discrepancy between the positions and momenta comes from the explicit update of positions in the Leapfrog scheme, while momenta updates depend on the learned potential energy. Energy conservation is ensured up to 0.2% relative difference between the initial and the final states on average due to the use of the numerical integrator.

Table 1: Averaged Wasserstein distance between the true and the generated samples at final time $t = 1.0$ evaluated on the test dataset.

	PDE-NHF	Baseline
Mean W_1 for positions	0.057	0.138
Mean W_1 for momenta	0.079	0.138

2. Second, **our model is capable of accurately interpolating** the dynamics throughout the entire trajectory without supervision at intermediate timesteps, see Table 2. By constraining the physics of the system, one prevents the model to learn arbitrary dynamics between the initial and the final conditions. The error slightly increases over time, due to a combination of numerical errors from the integrator and approximation errors from the learned Hamiltonian, resulting in progressive error amplification.

Table 2: Averaged Wasserstein distance between the true and the generated samples at intermediate times with PDE-NHF evaluated on the test dataset.

	$t = 0.2$	$t = 0.4$	$t = 0.6$	$t = 0.8$
Mean W_1 for positions	0.007	0.009	0.021	0.038
Mean W_1 for momenta	0.046	0.033	0.053	0.062

3. Third, **our model enables acceleration** of the integration process. Once trained, PDE-NHF is robust with respect to both the number of Leapfrog steps L and the integration timestep δt , as long as $L \times \delta t = T$. As shown in Table 3, using only $L = 5$ Leapfrog steps yields better results than the baseline. This property holds up to $L = 2$. In our experiments, integrating a system with the PIC method takes approximately 0.02 seconds. By contrast, using PDE-NHF with $L = 5$ steps achieves a twofold reduction in wall-clock time, making it a fast alternative as PIC methods are a reference for integrating such systems.

 Table 3: Averaged Wasserstein distance between the true and the generated samples at intermediate and final times with PDE-NHF evaluated on the test dataset. We used $L = 5$ and $\delta t = 0.2$.

	$t = 0.2$	$t = 0.4$	$t = 0.6$	$t = 0.8$	$t = 1.0$
Mean W_1 for positions	0.008	0.011	0.021	0.035	0.054
Mean W_1 for momenta	0.047	0.035	0.050	0.069	0.088

4. Finally, PDE-NHF **exhibits robustness** regarding the choice of the initial distribution. Due to the use of a translation-invariant potential network, it is insensitive to a shift in the initial position distribution. It is also able to extrapolate to different distribution scalings. For instance, after training with standard deviations comprised between 0.5 and 1.5 for both the positions and momenta distributions, PDE-NHF

yields correct results when sampling from initial distributions with standard deviations comprised between 1.5 and 2.5, as shown in Table 4. In particular, these results are better than the ones obtained with the baseline trained and evaluated on initial distributions with standard deviations comprised between 0.5 and 1.5.

Table 4: Averaged Wasserstein distance between the true and the generated samples at intermediate and final times when the initial position and momenta distributions for sampling (with standard deviation between 1.5 and 2.5) are different than the ones seen during training (with standard deviation between 0.5 and 1.5). We used $L = 25$ and $\delta t = 0.04$.

	$t = 0.2$	$t = 0.4$	$t = 0.6$	$t = 0.8$	$t = 1.0$
Mean W_1 for positions	0.015	0.036	0.061	0.083	0.092
Mean W_1 for momenta	0.068	0.108	0.122	0.099	0.118

We illustrate the capacity of PDE-NHF to integrate the 1D Vlasov-Poisson dynamics on $[0, T]$ on a test example in Figure 2. As can be seen from the cumulative histograms, the trained model may be seen as an accurate surrogate.

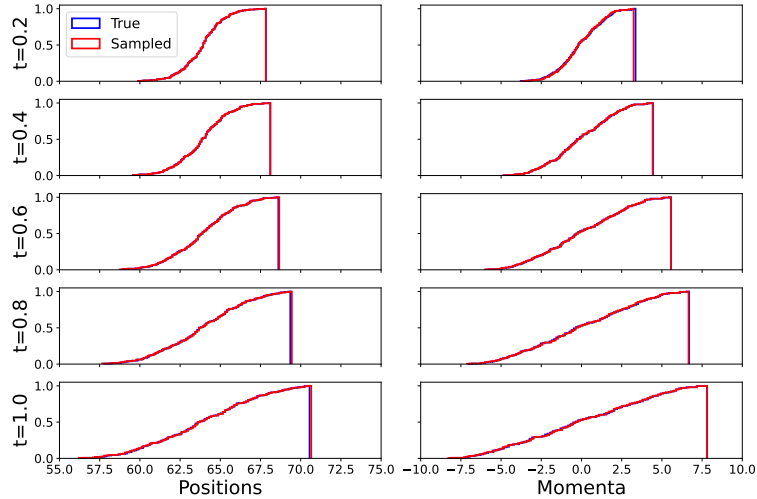


Figure 2: Normalized cumulative histograms from the true and the sampled trajectories for one test example, using $L = 25$ and $\delta t = 0.04$ for both training and sampling.

5. Conclusion

We introduced PDE-NHF, a Hamiltonian-based Normalizing Flows model designed to integrate systems governed by Hamiltonian kinetic equations. Leveraging a robust Leapfrog integration scheme, parametrized by a translation and permutation-invariant potential energy network, our generative model enables fast and accurate sampling of entire trajectories

in phase space. Through experiments on a 1D Vlasov-Poisson system, we demonstrated the model’s ability to faithfully reconstruct the true Hamiltonian dynamics from pairs of initial and final phase-space states. Promising directions include extending our methodology to higher-dimensional, real-world datasets. For example, by training on samples representing the early phase-space distribution of the universe and their evolved states obtained through the integration of a Vlasov-Poisson system, our approach could provide a fast, accurate, and compact surrogate model for gravitational processes, capturing the evolution from homogeneous initial conditions to today’s complex filamentary cosmic structure.

Acknowledgments

All authors are grateful to the Centre de Calcul Recherche et Technologie (CCRT) for providing computing and storage resources.

References

- Xavier Aguilar and Stefano Markidis. A deep learning-based particle-in-cell method for plasma simulations. In *2021 IEEE International Conference on Cluster Computing (CLUSTER)*, pages 692–697. IEEE, 2021.
- Vladimir Igorevich Arnold. *Mathematical methods of classical mechanics*, volume 60. Springer Science & Business Media, 2013.
- Francis Bernardeau, Stéphane Colombi, Enrique Gaztañaga, and Román Scoccimarro. Large-scale structure of the universe and cosmological perturbation theory. *Physics reports*, 367(1-3):1–248, 2002.
- C. K. Birdsall and A. B. Langdon. *Plasma Physics via Computer Simulation*. McGraw-Hill, 1991.
- Diego Del-Castillo-Negrete, Guannan Zhang, and Minglei Yang. A machine learning normalizing flow method to accelerate plasma kinetic simulations. In *APS Division of Plasma Physics Meeting Abstracts*, volume 2023 of *APS Meeting Abstracts*, page UP11.097, January 2023.
- Julien Derouillat, Arnaud Beck, Frédéric Pérez, Tommaso Vinci, M Chiaramello, Anna Grassi, M Flé, Guillaume Bouchard, I Plotnikov, Nicolas Aunai, et al. Smilei: A collaborative, open-source, multi-purpose particle-in-cell code for plasma simulation. *Computer Physics Communications*, 222:351–373, 2018.
- Laurent Dinh, David Krueger, and Yoshua Bengio. Nice: Non-linear independent components estimation, 2015. URL <https://arxiv.org/abs/1410.8516>.
- Ricardo A Fonseca, Luis O Silva, Frank S Tsung, Viktor K Decyk, Wei Lu, Chuang Ren, Warren B Mori, Shaogui Deng, Shiyoun Lee, T Katsouleas, et al. Osiris: A three-dimensional, fully relativistic particle in cell code for modeling plasma based accelerators. In *Computational Science—ICCS 2002: International Conference Amsterdam, The Netherlands, April 21–24, 2002 Proceedings, Part III 2*, pages 342–351. Springer, 2002.

- Herbert Goldstein, Charles Poole, and John Safko. *Classical Mechanics*. Addison Wesley, 3rd edition, 2002.
- Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.
- Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, and David Duvenaud. Scalable reversible generative models with free-form continuous dynamics. In *International Conference on Learning Representations*, volume 3, 2019.
- Samuel Greydanus, Misko Dzamba, and Jason Yosinski. Hamiltonian neural networks. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/26cd8ecadce0d4efd6cc8a8725cbd1f8-Paper.pdf.
- Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- R. W. Hockney and J. W. Eastwood. *Computer Simulation Using Particles*. CRC Press, 1988.
- Roger Williams Hockney, SP Goel, and JW Eastwood. Quiet high-resolution computer models of a plasma. *Journal of Computational Physics*, 14(2):148–158, 1974.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Lev D. Landau and Evgeny M. Lifshitz. *Mechanics*, volume 1 of *Course of Theoretical Physics*. Butterworth-Heinemann, Oxford, UK, 3rd edition, 1976. ISBN 978-0-7506-2896-9.
- Dwight Roy Nicholson and Dwight R Nicholson. *Introduction to plasma theory*, volume 1. Wiley New York, 1983.
- Michela Paganini, Luke de Oliveira, and Benjamin Nachman. Calogan: Simulating 3d high energy particle showers in multilayer electromagnetic calorimeters with generative adversarial networks. *Physical Review D*, 97(1):014021, 2018. doi: 10.1103/PhysRevD.97.014021.
- George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021. URL <http://jmlr.org/papers/v22/19-1028.html>.
- Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving

- nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 1530–1538, Lille, France, 07–09 Jul 2015. PMLR. URL <https://proceedings.mlr.press/v37/rezende15.html>.
- Danilo Jimenez Rezende, Sébastien Racanière, Irina Higgins, and Peter Toth. Equivariant hamiltonian flows. *arXiv preprint arXiv:1909.13739*, 2019.
- Andres C Rodriguez, Tomasz Kacprzak, Aurelien Lucchi, Adam Amara, Raphael Sgier, Janis Fluri, Thomas Hofmann, and Alexandre Réfrégier. Fast cosmic web simulations with generative adversarial networks. *Computational Astrophysics and Cosmology*, 5(1): 1–11, 2018.
- Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In Francis Bach and David Blei, editors, *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 2256–2265, Lille, France, 07–09 Jul 2015. PMLR. URL <https://proceedings.mlr.press/v37/sohl-dickstein15.html>.
- Vincent Souveton, Arnaud Guillin, Jens Jasche, Guilhem Lavaux, and Manon Michel. Fixed-kinetic neural Hamiltonian flows for enhanced interpretability and reduced complexity. In Sanjoy Dasgupta, Stephan Mandt, and Yingzhen Li, editors, *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238 of *Proceedings of Machine Learning Research*, pages 3178–3186. PMLR, 02–04 May 2024. URL <https://proceedings.mlr.press/v238/souveton24a.html>.
- Esteban G. Tabak and Eric Vanden-Eijnden. Density estimation by dual ascent of the log-likelihood. *Communications in Mathematical Sciences*, 8(1):217–233, 2010. ISSN 1539-6746. doi: 10.4310/CMS.2010.v8.n1.a11.
- Peter Toth, Danilo J. Rezende, Andrew Jaegle, Sébastien Racanière, Aleksandar Botev, and Irina Higgins. Hamiltonian generative networks. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=HJenn6VFvB>.
- J-L Vay, A Almgren, J Bell, L Ge, DP Grote, M Hogan, O Kononenko, R Lehe, A Myers, C Ng, et al. Warp-x: A new exascale computing platform for beam-plasma simulations. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, 909:476–479, 2018.
- Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J Smola. Deep sets. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS’17, page 3394–3404, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Baiyi Zhang, Guobiao Cai, Huiyan Weng, Weizong Wang, Lihui Liu, and Bijiao He. Physics-informed neural networks for solving forward and inverse vlasov–poisson equation via fully kinetic simulation. *Machine Learning: Science and Technology*, 4(4):045015, 2023.

Appendix A. Derivations of the loss functions

As Hamiltonian transformations are volume-preserving in phase space, the change of variables formula for the model joint distribution at time $T > 0$ for a Hamiltonian mapping $\mathcal{T}_\theta^T$ reads:

$$\Pi_{\theta,T}(q_T, p_T) = \Pi_0((\mathcal{T}_\theta^T)^{-1}(q_T, p_T)) \times 1.$$

In the case of Neural Hamiltonian Flows, one seeks to integrate over the momenta to recover the model positions distribution $\pi_{\theta,T}(q_T) = \int \Pi_{\theta,T}(q_T, p_T) dp_T$. The latter is intractable so one instead maximizes the following Evidence Lower Bound, derived from Jensen’s inequality (Toth et al., 2020). At fixed final position $q_T \in \mathbb{R}^d$:

$$\begin{aligned} \mathcal{L}_{\text{NHF}}(q_T; \theta) &= \mathbb{E}_{p_T \sim p_\theta(q_T)} [\ln \Pi_0((\mathcal{T}_\theta^T)^{-1}(q_T, p_T)) - \ln p_\theta(p_T | q_T)] \\ &= \mathbb{E}_{p_T \sim p_\theta(q_T)} \left[\ln \left(\frac{\Pi_0((\mathcal{T}_\theta^T)^{-1}(q_T, p_T))}{p_\theta(p_T | q_T)} \right) \right] \\ &= \mathbb{E}_{p_T \sim p_\theta(q_T)} \left[\ln \left(\frac{\Pi_{\theta,T}(q_T, p_T)}{p_\theta(p_T | q_T)} \right) \right] \\ &\leq \ln \int \frac{\Pi_{\theta,T}(q_T, p_T)}{p_\theta(p_T | q_T)} p_\theta(p_T | q_T) dp_T \\ &= \ln \pi_{\theta,T}(q_T). \end{aligned}$$

For PDE-NHF, since one works directly in phase space, there is no need to marginalize over the momenta and the Kullback-Leibler divergence between the target and the model distribution is directly tractable. At fixed condition $\mathbf{x} \in \mathcal{D}$:

$$\begin{aligned} \mathcal{L}_{\text{PDE-NHF}}(\mathbf{x}; \theta) &= \text{KL}(\Pi_T^\mathbf{x} || \Pi_{\theta,T}^\mathbf{x}) \\ &= \mathbb{E}_{(\mathbf{q}_T, \mathbf{p}_T) \sim \Pi_T^\mathbf{x}} [\ln \Pi_T^\mathbf{x}(\mathbf{q}_T, \mathbf{p}_T) - \ln \Pi_{\theta,T}^\mathbf{x}(\mathbf{q}_T, \mathbf{p}_T)] \\ &= -\mathbb{E}_{(\mathbf{q}_T, \mathbf{p}_T) \sim \Pi_T^\mathbf{x}} [\ln \Pi_{\theta,T}^\mathbf{x}(\mathbf{q}_T, \mathbf{p}_T)] + C \\ &= -\mathbb{E}_{(\mathbf{q}_T, \mathbf{p}_T) \sim \Pi_T^\mathbf{x}} [\ln \Pi_0^\mathbf{x}((\mathcal{T}_\theta^T)^{-1}(\mathbf{q}_T, \mathbf{p}_T))] + C, \end{aligned}$$

where C is a constant with respect to the model parameters and thus is not needed for training purpose.

Appendix B. Dataset generation

To build our datasets, we implemented a PIC method on a regular 1D grid made of 128 cells with box length 128. For generating sufficient diversity, the initial positions are drawn from a multivariate Normal distribution $\mathcal{N}(64, \sigma_q^2 I)$ and the initial velocities from another multivariate Normal distribution $\mathcal{N}(0, \sigma_p^2 I)$, with σ_q and σ_p random scalars between 0.5 and 1.5. This implies that the initial phase-space distribution is different for every example in the dataset. The simulation process follows the loop described below for $L = 25$ steps.

1. First, the density ρ on the grid is evaluated using the Cloud-In-Cell method ([Hockney et al., 1974](#)), a first-order interpolation scheme used in PIC simulations to map particle quantities onto a fixed grid. On the regular 1D grid with spacing δx , and grid points located at positions $x_i = i\delta x$, let us say we have a particle at position q_p , carrying a charge c_p . Let i be the index of the grid point to the left of q_p such that $x_i \leq q_p < x_{i+1}$. We define the normalized distance from the particle to the left grid point as

$$\delta = \frac{q_p - x_i}{\delta x}, \quad \text{with } 0 \leq \delta < 1.$$

The CIC weights assigned to the two nearest grid points are:

$$w_i = 1 - \delta = 1 - \frac{q_p - x_i}{\delta x}, \quad w_{i+1} = \delta = \frac{q_p - x_i}{\delta x}.$$

The particle's quantity c_p is distributed to the grid as follows:

$$\rho_i = \rho_i + \frac{c_p}{\delta x} \cdot w_i, \quad \rho_{i+1} = \rho_{i+1} + \frac{c_p}{\delta x} \cdot w_{i+1}.$$

Here, ρ_i represents the charge at grid point i , and the division by δx converts the assigned quantity into a density. All the particles have the same charge $c_p = 0.1$.

2. Then, the electrostatic Poisson equation is solved in Fourier space:

$$\frac{\partial^2 V}{\partial q^2} = -\frac{\rho}{\varepsilon_0} \longrightarrow \hat{V}_k = \frac{\hat{\rho}_k}{\varepsilon_0 k^2}.$$

Also,

$$F = -\frac{\partial V}{\partial q} \longrightarrow \hat{F}_k = -ik\hat{V}_k.$$

This yields:

$$\hat{F}_k = \frac{-ik\hat{\rho}_k}{\varepsilon_0 k^2}.$$

In our simulations, we assume that $\varepsilon_0 = 1$.

3. Finally, the system is evolved in phase space using one step of a symplectic Leapfrog scheme with integration timestep $\delta t = 0.04$ and periodic boundary conditions for the positions. Note that the initial conditions, the final time T and the size of the box are chosen such that no particle has crossed the boundaries in our examples.

Appendix C. Technical details about the generative models

For our experiments with PDE-NHF, the Deep Set architecture used to parametrize the potential energy function consists of two multilayer perceptrons (MLPs), ϕ and ψ , each containing a single hidden layer with 256 neurons and Softplus activations. The resulting model comprises approximately 130k learnable parameters. Training is performed using $L = 25$ Leapfrog steps with a fixed timestep $\delta t = 0.04$, over 200 epochs with random

minibatches of size 128, and a fixed learning rate of 3×10^{-4} using the Adam optimizer (Kingma and Ba, 2014). The full training procedure takes about 1 hour on an NVIDIA A100 GPU ©. Once trained, the model can perform sampling by drawing a Gaussian distribution of particles whose mean and covariance matrices are comparable to the ones seen during training and reversing the learned dynamics, that is reversing the timestep sign in the Leapfrog scheme.

For comparison, we also reported results obtained with a baseline model. This baseline consists of two MLPs, each with two hidden layers, totaling around 140k parameters when combining the two networks. The former are trained to directly map the initial particle states in phase space, sorted in ascending order as we found that it improved its performance, onto the final position distribution and the final momentum distribution, respectively. Note that such sorting is not needed for PDE-NHF. In this setting, the models act as transformations from the initial cumulative distribution function (CDF) in phase space onto each marginal final CDF, using the Mean Square Error loss function. We chose to employ a regression-based strategy since it is at the basis of many fundamental Machine Learning methods. Training the baseline model for 200 epochs under the same optimizer, minibatch size, and learning rate takes approximately 2 minutes on the same NVIDIA A100 GPU ©. The difference in training time comes from the unrolling of the Leapfrog steps for PDE-NHF.